Trial Task: CRM Prototype for a Glass Manufacturer

Oct 27, 2026 · @Seva

Overview

You will build a clickable frontend prototype of a CRM for a glass manufacturer and write a plan for its backend. If the result meets the criteria at the end of this sheet, we will hire you.

The task has two parts:

1. **Frontend prototype:** a working React + TypeScript app where all data lives in the browser.
2. **Backend and database plan:** a written architecture for how the same system would run for many companies. Planning only.

The most important rule: do not build a backend. No server, no database, no API of your own, no Supabase, no Firebase. The only external services you call from the browser are the OpenAI API (key provided by us) and Google Maps links. A backend in the prototype counts against you.

What we evaluate above all: is the UI reduced to what the user actually needs, and does it follow the user's real workflow as smoothly as possible? Think like a sales rep at a glass manufacturer who is on the road all day, often on a phone.

Part 1: Frontend prototype

Technical constraints

- React + TypeScript.
- All data lives in the browser (localStorage or IndexedDB). Ship it with realistic sample data: about 15 companies, 25 people, some meetings and tasks.
- No backend of any kind (see Overview).
- Works well on desktop and on mobile. Mobile is not an afterthought; sales reps use it in the car park after a meeting.
- The whole app is available in German and English, switchable in the UI.

- The OpenAI key is entered in a settings screen and stored in the browser, never committed to the repository.

1. Contacts: companies and people

- Two entities: **companies** and **people**.
- Many-to-many: a person can belong to one or more companies, a company can have one or more people.
- Each has standard fields (name, address, phone, email, role, etc.) plus free notes for any other information.
- Each company and each person has a list of **meetings**. A meeting stores date, participants and what was discussed.
- Inside a company or person, you can create a meeting and fill it by voice message.

2. Central voice command

A voice button available from anywhere in the app. The user speaks freely, for example: "I just met Anna Meier from Schreinerei Huber in Thun, she wants an offer for 12 shower glass panels, remind me to call her Friday."

1. User records, presses **Done**.
2. Audio is transcribed and interpreted via OpenAI.
3. The app shows a list of proposed actions: create company, create person, create meeting, create task, add notes to an existing record.
4. For each proposal the user can accept, edit or reject. Then everything accepted is saved.

It must recognise existing records (do not create a second Schreinerei Huber) and link new items correctly.

3. Tasks

- A tasks page with three columns: **Planned**, **In progress**, **Done**.
- A task can be linked to a company and/or a person, and shows up on their page.
- Tasks can also be created through the central voice command.

4. Knowledge base

- Sections such as **Installations** and others you find useful.

- Search for a product and open its installation tutorial (step-by-step text with example images; placeholder images are fine).
- A voice button where a technician can speak their know-how; it is turned into a structured knowledge entry, shown for review before saving.
- A **chat assistant** (OpenAI) that answers any question. It should use the CRM data and the knowledge base as context where relevant.

5. Email inbox (mockup)

- An inbox with fake sample emails, no real email connection.
- Some emails are quote requests. On those, a button extracts the request and shows proposed data (customer, products, quantities, dimensions).
- After the user confirms, a button **Transfer to [ERP name to be confirmed]** simulates handing the quote over.

6. Companies on a map and route planning

- In the companies tab, the user can show companies on a map.
- The user selects several companies and presses **Plan route**.
- The app orders the stops into an efficient route (you choose the approach, explain it in your README) and opens it in Google Maps with one tap, ready for navigation on the phone.

7. Lead search (mockup only, not functional)

- The user enters criteria: region, type of business (carpentry shops, architecture firms, etc.) and further filters.
- Pressing **Search** shows a believable result list, as if companies had been found in the Swiss commercial register and their websites checked.
- Static sample results are enough. Do not build the search.

8. simap Bot tab

- Just an empty tab with this name in the navigation. Nothing to build.

Part 2: Backend and database architecture (plan only)

Do not implement any of this. Write it down.

Plan how the CRM from Part 1 would run as a real product used by many companies at once. Target: **50 companies with 5 to 10 employees each**, so roughly 250 to 500 users. Each company sees only its own data.

You decide the architecture. It should make sense for a product of this size, not for a hypothetical million users. Justify your choices.

Your plan should cover:

- **Overall architecture:** components and how they talk to each other, as a diagram.
- **Database schema:** all tables, key fields and relations (including the many-to-many between companies and people, meetings, tasks, knowledge entries, emails), as an ER diagram.
- **Multi-tenancy:** how data is separated between customer companies, and why you chose that approach.
- **Authentication and roles:** login, users per company, admin vs. normal user.
- **AI features:** where the OpenAI calls happen, how the API key is protected, how voice files are handled and stored.
- **Integrations:** how email inbox, ERP transfer, Google Maps and the lead search would connect.
- **Hosting and operations:** where it runs, backups, rough monthly cost, data location (Swiss customers, data protection).
- **What you would build first** and what you would deliberately leave out at this scale.

Deliverables, timeline and evaluation

What you hand in

- [ ] Link to a Git repository with the prototype
- [ ] A deployed version we can open on desktop and phone (e.g. Vercel)
- [ ] README: how to run it, what you built, what you left out and why, how the route ordering works
- [ ] Architecture plan as a document (PDF or Markdown) with the diagrams
- [ ] A short screen recording (max. 5 minutes) walking through the main workflow

Timeline

- Deadline: **Tuesday,** Nov 3, 2026

- Expected effort: **about 10 hours**
- Payment: **INR 860 per hour**, up to 10 hours (same hourly rate as our team)

It does not have to be perfect or complete by the deadline. Prioritise: a polished core workflow (contacts, voice command, tasks) beats every feature half done. List what is missing in your README.

How we evaluate

Criterion	What we look for
UI/UX focus	Only what the user needs on each screen, workflow follows the real sales process, few clicks, works one-handed on a phone
Voice workflow	Free speech becomes correct, linked proposals; reviewing and correcting is fast and clear
Rules followed	No backend in the prototype, no backend code in Part 2, all features covered
Code quality	Clean TypeScript, sensible component structure, readable
Architecture	Fits 50 companies x 5 to 10 users, clear tenant separation, choices justified, not over-engineered
Communication	README and plan are short, precise and honest about what is missing

If something is unclear, ask before you start building. Asking good questions is part of the task.